



JAVASCRIPT EVENTS & ASYNC PROGRAMMING

GV: Tạ Chí Hiếu

Mục tiêu bài học

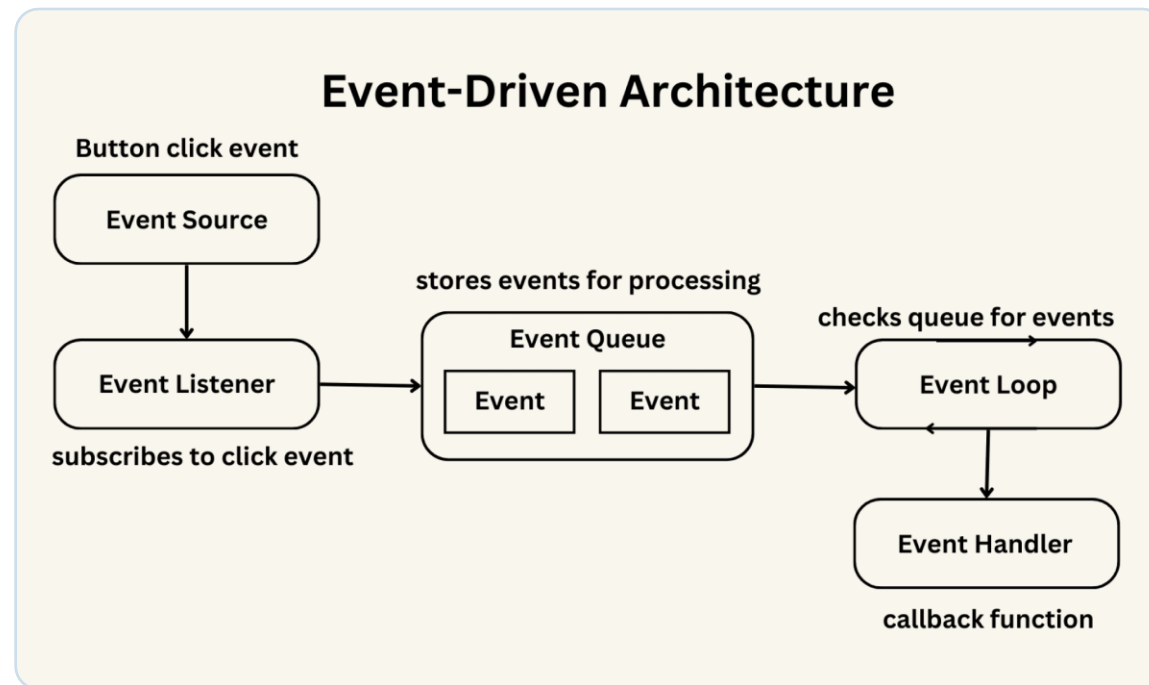
- ☑ • Thành thạo **Event Handling** để tương tác với người dùng
 - Hiểu sâu về **Event Propagation** (Bubbling và Capturing)
- ☑ • Áp dụng **Event Delegation** để tối ưu hóa hiệu suất
- ☑ • Nắm vững **Promises** và cú pháp **Async/Await** hiện đại
 - Sử dụng thành thạo **Fetch API** để làm việc với dữ liệu từ Server
- ☑
- ☑

Events là gì?

Định nghĩa: Events là các hành động (actions) hoặc sự kiện (occurrences) xảy ra trong trình duyệt.

- **User events:** Click, type, scroll, hover...
- **System events:** Load, resize, error, online/offline...

JavaScript thực hiện việc "*lắng nghe*" (listens) và "*phản hồi*" (responds) lại các sự kiện này để tạo ra sự tương tác.



Event Flow: User Action → Event Triggered → Event Listener → Event Handler

Event Types - Mouse Events

Các sự kiện chuột phổ biến:



- **click**: Click chuột trái
- **mouseenter / mouseleave**: Di chuột vào/ra
- **mousedown / mouseup**: Nhấn/thả chuột

```
// Cú pháp: element.addEventListener(event, handler)
element.addEventListener('click', handler);
element.addEventListener('mouseenter', handler);
element.addEventListener('mouseleave', handler);
```

Event Types - Keyboard Events

Theo dõi tương tác từ bàn phím:



- **keydown**: Khi phím được nhấn xuống
- **keyup**: Khi phím được thả ra
- ~~keypress~~: (Đã bị loại bỏ - Deprecated)

Thường dùng cho: Phím tắt, điều khiển game, form input.

```
element.addEventListener('keydown', (e) => {  
    console.log(`Key pressed: ${e.key}`);  
});
```

Event Types - Form Events

Quản lý tương tác trong Biểu mẫu:



- **submit**: Gửi form (sự kiện quan trọng nhất)
- **change**: Khi giá trị input thay đổi và mất focus
- **input**: Thay đổi giá trị ngay lập tức khi gõ
- **focus / blur**: Khi element được tập trung / mất tập trung

```
form.addEventListener('submit', (e) => {  
    e.preventDefault(); // Ngăn trang load lại  
    console.log('Form submitted');  
});
```

Event Object

Thông tin chi tiết về sự kiện:

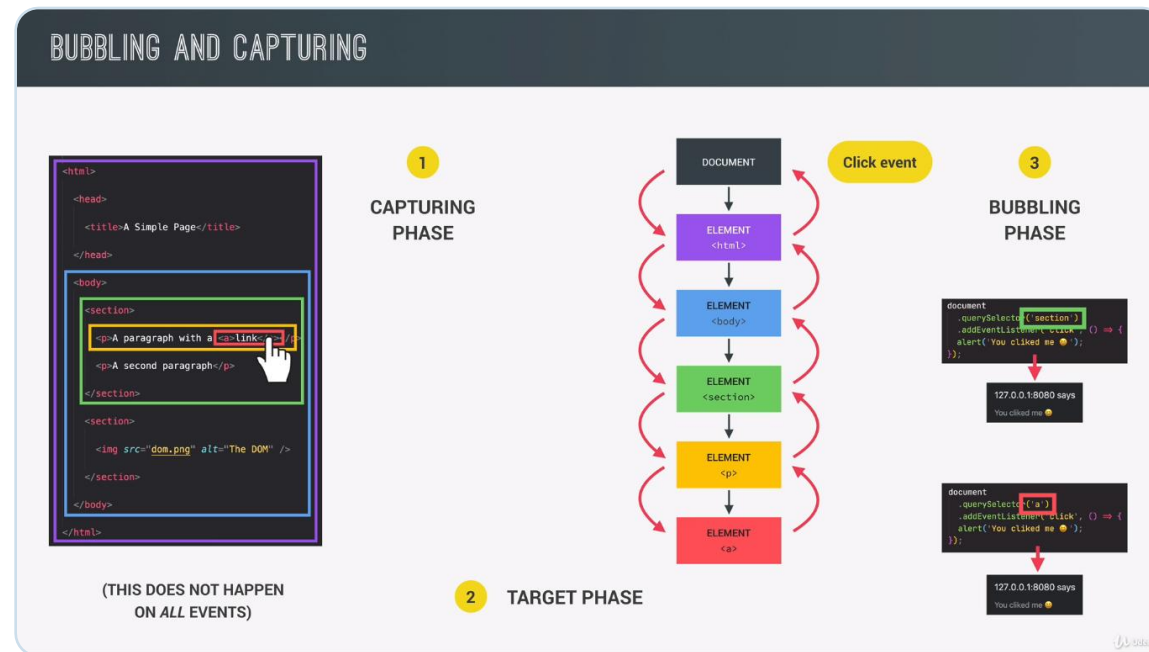
- **event.type**: Tên của sự kiện (VD: 'click')
- **event.target**: Element trực tiếp gây ra sự kiện
- **event.preventDefault()**: Chặn hành vi mặc định của trình duyệt
- **event.stopPropagation()**: Ngừng lan truyền sự kiện

```
button.addEventListener('click',  
function(event) {  
    // Log loại sự kiện  
    console.log(event.type); // 'click'  
    // Element được nhấn  
    console.log(event.target);  
    // Ngăn chặn mặc định  
    event.preventDefault();  
});
```

Event Propagation - Bubbling

Khái niệm: Sự kiện "nổi bọt" từ element con lên đến các element cha.

- Là hành vi mặc định trong DOM.
- Luồng: **Child → Parent → Grandparent → Document**.
- Dễ dàng quản lý nhiều con thông qua cha.



"When an event happens on an element, it first runs the handlers on it, then on its parent, then all the way up on other ancestors."

Event Propagation - Capturing

Khái niệm: Sự kiện lan truyền từ gốc xuống đến element đích.

- Xảy ra **trước** quá trình Bubbling.
- Luồng: **Root → Parent → Child** (Top-Down).
- Kích hoạt bằng cách đặt tham số thứ 3 là **true** hoặc **{ capture: true }**.

[Window / Document]



Parent Element



Target Element

Capturing Phase (Downwards)

```
// Lắng nghe sự kiện ở giai đoạn Capturing  
element.addEventListener('click', handler, true);
```

Event Delegation

Kỹ thuật: Gán một listener duy nhất cho cha để quản lý tất cả các con.

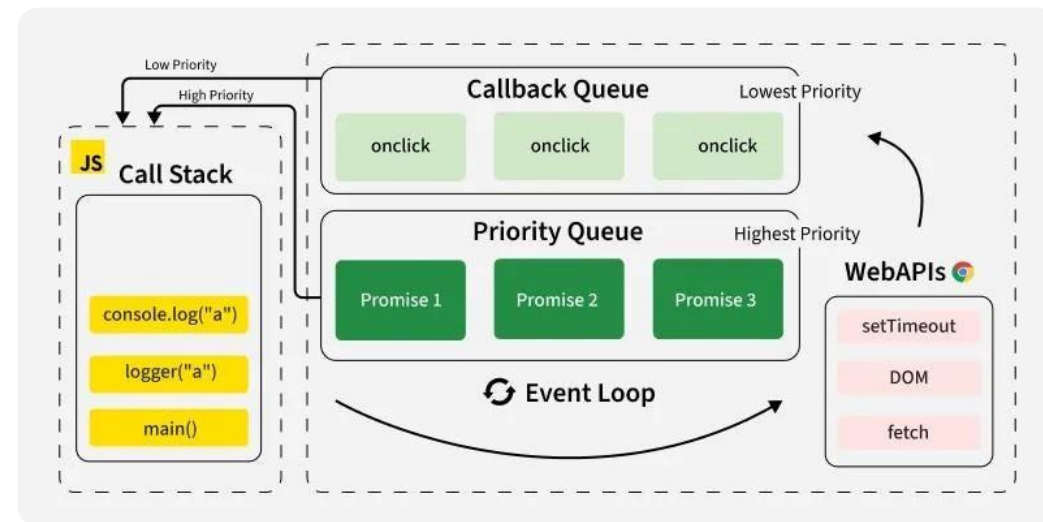


- Dựa trên cơ chế **Bubbling**.
- **Tối ưu hiệu suất:** Không cần tạo hàng trăm listener.
- Làm việc tốt với các **phần tử động** (mới được thêm).

```
// Listener gán cho 'parent-list'  
document.getElementById('list').addEventListener('click', (e) => {  
    if (e.target.tagName === 'LI') {  
        console.log('List item clicked');  
    }  
});
```

Asynchronous Programming - Concept

- JavaScript là **Single-threaded** (đơn luồng).
- Các tác vụ bất đồng bộ (Network, Timer) không gây **block** luồng chính.
- Event Loop** là cơ chế điều phối việc thực thi các tác vụ này.
→ Giúp giao diện luôn mượt mà trong khi dữ liệu đang được tải.



Synchronous:

Task A → Task B → Task C
(Chờ đợi lần lượt)

Asynchronous:

Task A (Async) → Start Task B
(Thực hiện song song, không chờ đợi)

setTimeout

Hàm bất đồng bộ cơ bản nhất:

- Thực thi code sau một khoảng delay.
- **Không gây block** luồng chính.
- Trả về *timer ID* (có thể dùng để hủy bằng `clearTimeout`).

```
console.log('Start');  
setTimeout(() => {  
  console.log('Async operation');  
}, 1000);  
console.log('End');  
// Output: Start -> End -> Async operation
```

Promises - Concept

Đối tượng đại diện cho sự hoàn thành (hoặc thất bại) của một tác vụ bất đồng bộ trong tương lai.

- **Giải quyết:** "Callback Hell" (lồng nhau quá nhiều).
- **Dễ đọc:** Cấu trúc code tuần tự hơn.

3 Trạng thái của Promise:

1. **Pending:** Đang chờ xử lý



2. **Fulfilled:** Thành công (Resolve)

3. **Rejected:** Thất bại (Reject)

Promises - Creating Promise

Cú pháp khởi tạo:

new Promise((resolve, reject) => { ... })

- Gọi **resolve(value)** khi tác vụ thành công.
- Gọi **reject(error)** khi có lỗi xảy ra.

```
const myPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    const success = true;
    if (success) {
      resolve('Operation Successful!');
    } else {
      reject('Something went wrong');
    }
  }, 1000);
});
```

Promises - Using Promise

Các phương thức tiêu thụ Promise:

- **.then()**: Xử lý khi thành công (value).
- **.catch()**: Xử lý khi có lỗi (error).
- **.finally()**: Luôn chạy sau cùng (dọn dẹp).

Có khả năng **Chaining** (chuỗi) để xử lý liên tiếp.

```
myPromise
  .then(result => {
    console.log(result);
    return nextStep(result);
  })
  .then(nextResult => {
    console.log(nextResult);
  })
  .catch(error => {
    console.error(error);
  })
  .finally(() => {
    stopLoading();
  });
```

Async/Await (ES2017) - Concept

Cách viết code bất đồng bộ trông giống như **đồng bộ** (synchronous), dễ đọc và bảo trì hơn.

→ Là "Syntactic sugar" (cú pháp bọc ngoài) của Promises.

Promise Chain:

```
getData()  
.then(res => {  
    return res.json();  
})  
.then(data => {  
    console.log(data);  
});
```

Async/Await Style:

```
async function handle() {  
    const res = await getData();  
    const data = await res.json();  
    console.log(data);  
}
```


Async/Await - Example

```
async function fetchUser() {  
  try {  
    // Luồng xử lý tuần tự, cực kỳ dễ hiểu  
    const response = await fetch('/api/user');  
    const user = await response.json();  
    console.log(user);  
  } catch (err) {  
    // Xử lý lỗi tập trung tại đây  
    console.error('Fetch failed:', err);  
  }  
}
```

Readable flow, centralized error handling with try/catch.

Fetch API - Basic

Giao diện gốc của trình duyệt để gọi tài nguyên mạng:

- Dựa trên **Promises**.
- Thay thế hoàn toàn *XMLHttpRequest* (AJAX) cũ kỹ.
- Sạch sẽ, mạnh mẽ và linh hoạt.

```
fetch('https://api.example.com/data')
  .then(res => {
    // Phải parse kết quả
    return res.json();
  })
  .then(data => {
    console.log(data);
  });
```

Fetch API - With Async/Await

Sự kết hợp hoàn hảo - **The cleanest way to fetch.**

```
async function getData() {  
    const res = await fetch('url/api/data');  
    const data = await res.json();  
    return data;  
}  
// Các bước được trình bày rõ ràng, dễ đọc.
```

Error Handling - try/catch

Xử lý lỗi an toàn cho code bất đồng bộ:

- **try { ... }**: Chứa đoạn code có thể gây lỗi.
- **catch (error) { ... }**: Nơi xử lý khi lỗi xảy ra.
- **finally { ... }**: Luôn thực thi (dọn dẹp tài nguyên).

```
try {  
    await riskyOperation();  
} catch (error) {  
    handleError(error);  
} finally {  
    cleanup();  
}
```

THỰC HÀNH - Events



Bài tập yêu cầu:

1. Thêm *click event listener* cho một nút bấm.
2. Thêm *keyboard event listener* để nhận diện phím nhấn.
3. Xử lý sự kiện *submit* của form và ngăn load lại trang.
4. Kiểm tra sự lan truyền sự kiện (*bubbling*) qua các thẻ lồng nhau.
5. Triển khai *event delegation* cho một danh sách (UL/LI).

THỰC HÀNH - Promises



Bài tập yêu cầu:

1. Tự tạo một *Promise* mô phỏng việc lấy dữ liệu (delay 2s).
2. Sử dụng *.then()* và *.catch()* để xử lý kết quả.
3. Thực hiện chuỗi *Chain promises* qua nhiều bước xử lý.
4. Mô phỏng lỗi và xử lý tập trung bằng *.catch()*.
5. Chuyển đổi một hàm *callback* truyền thống sang dạng *Promise*.

THỰC HÀNH - Async/Await & Fetch



Bài tập yêu cầu:

1. Chuyển đổi chuỗi *Promise* sang cú pháp *Async/Await*.
2. Sử dụng *Fetch API* để gọi một API công khai (VD: JSONPlaceholder).
3. Quản lý lỗi kết nối bằng khối *try/catch*.
4. Lấy dữ liệu người dùng từ API và xử lý dữ liệu JSON.
5. Hiển thị dữ liệu đã lấy được lên giao diện (DOM).

TỔNG KẾT



- **Events:** Click, submit, keyboard events là nền tảng tương tác.
- **Event Propagation:** Cơ chế Bubbling và Capturing trong DOM.
- **Event Delegation:** Tối ưu hiệu suất cho dynamic content.
- **Promises:** Quản lý các tác vụ bất đồng bộ hiệu quả.
- **Async/Await:** Cách tiếp cận hiện đại để xử lý async code.
- **Fetch API:** Gửi và nhận dữ liệu từ server dễ dàng.
- **Error Handling:** Luôn dùng try/catch để đảm bảo ứng dụng an toàn.

HỎI ĐÁP

- ? Bubbling vs Capturing khác gì?
- ? Event delegation dùng khi nào?
- ? Promise vs Callback khác gì?
- ? Async/await vs Promise chains?
- ? Fetch API có thể dùng cho POST không?



BÀI HỌC TIẾP THEO

Nội dung chính: Modules, Classes, Clean Code, Project Organization.

